Lovelace.Proofs — The Equations and Their Proofs

> Math is LaTeX (`$ … $` , `$$ … $$`); Lean identifiers are in backticks.

Setup

Base `b : Nat` , `b ≥ 2` . Digits are stored **least-significant first**. `ofDigits` evaluates a digit list (Horner form):

$$[c_0, c_1, \ldots, c_n] \mapsto \sum_{i=0}^{n} c_i\, b^i.$$

`div` is floor division, `mod` the remainder, linked by the workhorse identity

$$x = (x \bmod b) + b \cdot (x \operatorname{div} b).$$

---

1. Representation

Equations

$$\operatorname{digits}_b(n) = (n \bmod b) \;::\; \operatorname{digits}_b(n \operatorname{div} b)$$

$$c_i = \left(n \operatorname{div} b^i\right) \bmod b.$$

Theorems (for `2 ≤ b`)

```
ofDigits_digits  : ofDigits b (digits b n) = n
digits_lt_base   : ∀ d ∈ digits b n, d < b
digits_getDigit  : getDigit (digits b n) i = (n / b^i) % b
digits_injective : Function.Injective (digits b)
```

Proof. *Strong induction on* `n` . For `n ≠ 0` :

```
ofDigits (digits n)
  = ofDigits ((n mod b) :: digits (n div b))
  = (n mod b) + b · ofDigits (digits (n div b))
  = (n mod b) + b · (n div b)          -- IH on n div b < n
  = n                                   -- Nat.mod_add_div
```

Extraction is induction on `i` using `(n/b)/b^i = n/b^(i+1)` ; uniqueness follows by applying `ofDigits` to both sides of `digits n = digits m` .

---

2. Addition (carry)

Equations

$$\operatorname{carry}_0 = 0, \qquad e_i = (c_i + d_i + \operatorname{carry}_i) \bmod b, \qquad \operatorname{carry}_{i+1} = (c_i + d_i + \operatorname{carry}_i) \operatorname{div} b.$$

Theorem — value preservation

$$\sum_i e_i\, b^i \;+\; \operatorname{carry} \cdot b^{\ell} \;=\; \sum_i c_i\, b^i \;+\; \sum_i d_i\, b^i \;+\; k.$$

```
addDigits_correct :
  ofDigits b (addDigits b cs ds k).1 + (addDigits b cs ds k).2 · b^len
    = ofDigits b cs + ofDigits b ds + k
```

Proof. *Induction on the digit list.* One column step:

```
(x+y+k) mod b + b · (ofDigits rest + carry · b^n)
  = x + b·ofDigits xs + y + b·ofDigits ys + k
```

After distributing `b` and applying the IH, this reduces to `(x+y+k) mod b + b·((x+y+k) div b) = x + y + k` — again `Nat.mod_add_div`.

---

3. Subtraction (borrow)

Equations (borrow flag `∈ {0,1}`)

$$\text{borrow}_0 = 0, \qquad e_i = (b + c_i - d_i - \text{borrow}_i) \bmod b, \qquad \text{borrow}_{i+1} = 1 - (b + c_i - d_i - \text{borrow}_i) \operatorname{div} b.$$

Theorem — invariant as a sum (no truncated `-`)

$$\sum_i c_i\, b^i \;+\; \text{borrow}_{\text{out}} \cdot b^{\ell} \;=\; \sum_i d_i\, b^i \;+\; \text{borrow}_{\text{in}} \;+\; \sum_i e_i\, b^i.$$

```
subDigits_correct :
  ofDigits b cs + borrowOut · b^len
    = ofDigits b ds + borrowIn + ofDigits b result

subDigits_no_borrow :
  ofDigits b ds ≤ ofDigits b cs →
    borrowOut = 0 ∧ ofDigits result = ofDigits b cs - ofDigits b ds
```

Proof. *Induction on the digit list.* The heart is the single-column lemma

```
subColumn : x + b·(1 - t div b) = y + borrow + (t mod b)
            where t = b + x - y - borrow
```

Its proof: `b·(1 - t div b) = b - b·(t div b)` and `t mod b = t - b·(t div b)`, so both sides reduce to `x + b = y + borrow + t` — true because `t` *is* `b + x - y - borrow`.

For the no-borrow case: if `borrowOut ≥ 1`, the invariant's left side is `≥ ofDigits cs + b^len`, while its right side is `ofDigits ds + ofDigits result < ofDigits ds + b^len ≤ ofDigits cs + b^len` (using `ofDigits result < b^len`) — a contradiction.

---

4. Multiplication (convolution + carry)

Equations

$$s_n = \sum_{k=0}^{n} c_k\, d_{n-k}, \qquad e_n = (s_n + \text{carry}_{n-1}) \bmod b, \qquad \text{carry}_n = (s_n + \text{carry}_{n-1}) \operatorname{div} b.$$

Cauchy product — the convolution is the coefficient of the product:

$$\Big(\sum_i c_i b^i\Big)\Big(\sum_j d_j b^j\Big) \;=\; \sum_n \Big(\sum_{k=0}^{n} c_k\, d_{n-k}\Big) b^n.$$

Theorems

```
ofDigits_conv     : ofDigits b (conv cs ds) = ofDigits b cs · ofDigits b ds
normalize_correct : (∀ d ∈ normalize b s, d < b) ∧ ofDigits b (normalize b s) = ofDigits b s
mulDigits_correct : ofDigits b (normalize b (conv cs ds)) = ofDigits b cs · ofDigits b ds
```

Proof. The Cauchy product is induction on `cs`; one step is one distributivity:

```
ofDigits (conv (c::cs) ds)
  = c·ofDigits ds + b·ofDigits (conv cs ds)          -- addLists + map_mul lemmas
  = c·ofDigits ds + b·(ofDigits cs · ofDigits ds)    -- IH
  = (c + b·ofDigits cs) · ofDigits ds
```

The carry pass uses the invariant `ofDigits (normalizeAux s c) = ofDigits s + c` (the running carry is added into the units column); the final carry is emitted as its own base-`b` digits. `mulDigits_correct` composes the two.

---

5. Division (long division)

Digit form — the running-remainder recurrence (the computation). One column needs only the running remainder `r` (`r < d`) and the current dividend digit c_i (`c_i < b`):

$$t_i = r_i \cdot b + c_i, \qquad q_i = t_i \operatorname{div} d, \qquad r_{i+1} = t_i \bmod d, \qquad r_0 = 0.$$

By `divColumn`, `q_i < b` and `r_{i+1} < d` — the state never leaves the `(b, d)`-bounded range, so the loop runs on `r` and c_i alone (no whole-number value is materialized), exactly like the carry/borrow columns of add/sub/mul.

Theorems (divisor `0 < d`; correctness `a = q·d + r`, `0 ≤ r < d`, `0 ≤ q_i < b`)

```
divColumn                 : r < d → c < b → (r·b + c) / d < b ∧ (r·b + c) % d < d   -- one column, small state
divDigitsMSB_correct      : ofDigitsMSB b (divDigitsMSB …).1 · d + (…).2 = ofDigitsMSB b ds + r · b^len
divDigits_quotient_lt_base : ∀ q ∈ (divDigits b ds d).1, q < b
divDigits_remainder_lt    : (divDigits b ds d).2 < d
divDigits_correct         : ofDigits b (divDigits b ds d).1 · d + (divDigits b ds d).2 = ofDigits b ds
divDigits_quotient_eq_div : ofDigits b (divDigits b ds d).1 = ofDigits b ds / d
divDigits_remainder_eq_mod : (divDigits b ds d).2 = ofDigits b ds % d
divDigits_getDigit        : getDigit (divDigits b ds d).1 i = (ofDigits b ds / d / b^i) % b   -- derived extraction
```

Proof. *Induction over the MSB-first digit list*, maintaining the running-remainder invariant

```
ofDigits (processed prefix) = ofDigits (quotient so far) · d + r,   0 ≤ r < d
```

The closed-form extraction `divDigits_getDigit` is a *consequence* (the head/tail fold of `ofDigits`, induction on `i`, like Representation's `digits_getDigit`); the algorithm itself is the recurrence, not that closed form. Because `r < d` and `c_i < b`, `r·b + c_i < d·b`, so `q_i = (r·b + c_i) div d < b` (no normalization needed). The final invariant is `a = q·d + r` with `r < d` — the digit-by-digit computation of `Nat.div` / `Nat.mod`.

Summary

Operation	Equation	Key lemma	Proof
Representation	`c_i = (a div b^i) mod b`	`Nat.mod_add_div`	strong induction
Addition	`e_i = (c_i+d_i+carry_i) mod b`	`Nat.mod_add_div`	list induction
Subtraction	`e_i = (b+c_i−d_i−borrow_i) mod b`	`subColumn`	list induction
Multiplication	`e_n = (Σ c_k d_{n−k} + carry) mod b`	`Nat.mul_add`	Cauchy product + carry
Division	`q_i = (r_i·b + c_i) div d`	`a = q·d + r, 0 ≤ r < d`	MSB-first fold